

Interface-Scoped Dispatch: General Delegation with $O(1)$ Self-Calls

ANONYMOUS AUTHOR(S)

The conceptual appeal of object composition is its ability to build loosely coupled systems from independent runtime components. Yet conventional languages prevent composition from fully realizing this potential, due to the absence of late-bound self-calls (open recursion), a defining strength of implementation inheritance. Filling this gap while fully retaining the properties of object composition remains an unmet challenge.

We present *Parts*, a general delegation model that meets this challenge by unifying independent runtime components, arbitrary composition, and open recursion. Its enabling mechanism, interface-scoped dispatch (ISD), assigns composite identity per interface at construction, allowing self-calls to dynamically dispatch through overriding composites. *Parts* is realized as a compiler plugin for Java 8–27 with IDE integration. It achieves an $O(1)$ dispatch cost that is empirically equivalent to a standard vtable call, offering open recursion through composition without compromise.

CCS Concepts: • **Theory of computation** → **Object oriented constructs**; • **Software and its engineering** → **Polymorphism**; **Inheritance**.

Additional Key Words and Phrases: object composition, delegation, open recursion, self-calls, dynamic dispatch, interface-scoped dispatch, mixins, traits, inheritance, Java

ACM Reference Format:

Anonymous Author(s). 2026. Interface-Scoped Dispatch: General Delegation with $O(1)$ Self-Calls. 1, 1 (October 2026), 23 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Object composition is the standard prescription for structuring behavior from reusable components [1]. Most object-oriented languages provide varying levels of composition through *forwarding* [16], offering an alternative to implementation inheritance and its well-known hazards such as fragile base classes and coupling to superclass internals. But forwarding gives up a vital property that makes inheritance practical: *open recursion*. With open recursion, a component’s self-calls remain late-bound, allowing them to reach overrides supplied by forwarding composites. Without it, self-calls are fixed to the component’s local implementation and never reach composite overrides. Yet modern languages provide no practical way to retain this behavior.

The challenge is to retain the architectural benefits of object composition while recovering open recursion. A comprehensive resolution must preserve *independent components* (late-bound objects supplied at construction) and *arbitrary composition* (unrestricted composition topologies), while providing *open recursion* (self-calls late-bound through the composite). Prior composition models achieve at most two of these three properties (Section 7).

The conventional object composition of conventional languages differs in implementation but reduces to basic forwarding. Some, like Rust and Swift, require manual forwarding; others, such as Kotlin’s by clause [29] and Go’s struct embedding [31], provide it automatically. They all support independent components under arbitrary composition, but forwarding alone does not preserve

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/10-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

the composite identity needed for open recursion: a component's this remains the component, so its self-calls cannot be redirected through the composite. This is the *Object Schizophrenia* problem [4, 16].

Trait composition, including Scala's with [11] and mixins [5], makes the opposite trade. Self-calls through this dispatch in the composed hierarchy, so overrides remain visible. But traits are fused into the hierarchy at compile time and execute as a single runtime object. Components lose independent runtime identity, and composition is structurally static and cannot be defined at runtime.

Several research projects (Section 7) have explored *delegation* [3, 15]: forwarding models that attempt to restore open recursion. So far, these models support only restricted composition topologies rather than the arbitrary topologies that ordinary forwarding admits (Section 3.3). Most introduce auxiliary mechanisms for maintaining a level of composite identity. These mechanisms tend to increase overall implementation complexity and, in some cases, incur substantial runtime overhead.

As a general delegation model, Parts resolves these historical limitations by unifying independent runtime components, arbitrary composition, and open recursion. Its enabling mechanism, *interface-scoped dispatch* (ISD), uses *links* to assign composite identity per interface at construction to *parts*, components that participate in delegation. A part's self-calls then dispatch dynamically through the composites that link it. This per-interface identity is the core principle that enables dispatch across arbitrary composition topologies while preserving independent runtime objects. Links work with both part and non-part components, supporting delegation with parts and forwarding with non-parts, allowing both to coexist in the same composite. Parts is realized as a compiler plugin for Java 8–27 with IDE integration; its execution cost is $O(1)$, empirically equivalent to a standard vtable call.

The core contributions are:

- **Parts:** a general delegation model in which components are independent runtime objects, composable arbitrarily under partial delegation, with self-calls dispatching through the composite and interface contracts statically enforced. The model is defined over standard interface hierarchies with a central dispatch invariant (Invariant 1) and composite-scoped encapsulation (@internal).
- **Interface-scoped dispatch (ISD):** the mechanism that assigns composite identity per interface at construction, enabling open recursion under arbitrary composition via partial delegation (where a composite claims a proper subset of a part's interfaces).
- **An implementation:** a compiler plugin for Java 8–27 realizing the model with $O(1)$ self-dispatch cost, with open-source release [33] and IDE integration.

2 Overview

Consider a minimal example that shows @part and @link working in a concrete single-rooted composite.

```
interface Actor {
    void takeAction();
    void attack();
}

@part class Hero implements Actor {
    public void takeAction() { attack(); }
    public void attack() { println("Swing club!"); }
}
```

```
class Wizard implements Actor {
    @link Actor actor;
    Wizard(Actor actor) { this.actor = actor; }
    public void attack() { println("Cast spell!"); }
}

new Wizard(new Hero()).takeAction(); // "Cast spell!"
```

When Wizard is constructed, the compiler wires Hero's composite identity for Actor to Wizard. A call to takeAction() dispatches to Hero.takeAction(), and the self-call attack() within it dispatches back through Wizard, invoking Wizard.attack(), not Hero.attack().

This is the decorator pattern with one addition: the wrapped object's self-call routes back through the composite. A plain decorator would forward takeAction() to the Hero, where the attack() call would stay on Hero ("Swing club!"), losing the override.

Because the link field accepts any Actor at construction time, the part is a real late-bound object, not fused into the class hierarchy. A different Actor implementation can be substituted without changing Wizard.

Interface contracts are enforced at compile time. The link field has type Actor, and the compiler verifies that Wizard implements the delegated interface. A raw part may not escape the composite dispatch path: passing or assigning this as the concrete part type is a compile error; interface or Object types are required (Section 4.5).

This example is single-rooted: Wizard is the sole composite for Hero, and one composite identity suffices. The following section exposes the limit of the single-identity model and the per-interface generalization that resolves it.

3 The Model

3.1 Motivating example

The following example shows a part with interfaces rooted at different composites. Hero implements both Combatant and Actor, and two wrappers decorate it: a Wizard changes how it fights (Combatant), while a Demon partially possesses it, seizing its name and will (Actor) but leaving its trained combat untouched. As in the overview, the wrapped hero's self-calls route back through the wrappers; now two wrappers claim different interfaces of the same part. So from a single Hero instance, attack() must dispatch to Wizard and name() to Demon. A single composite identity cannot serve both. Assigning a separate composite identity per interface resolves this. Figures 1 and 2 show the structure.

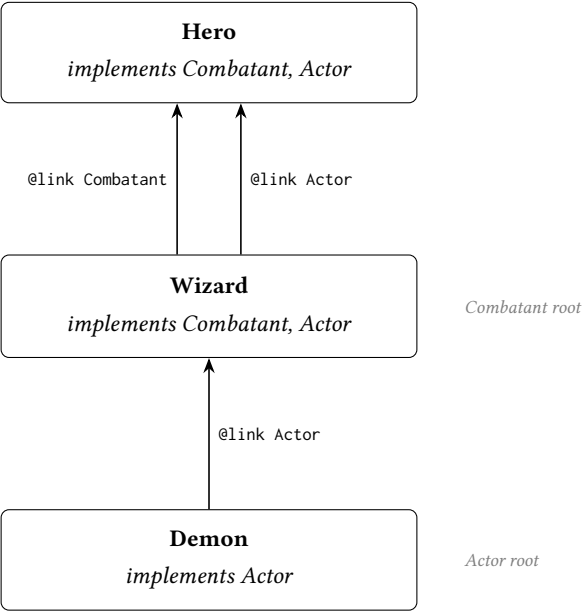


Fig. 1. Composition structure. Arrows show link fields labeled by the delegated interface. Wizard is the root for Combatant dispatch; Demon is the root for Actor dispatch.

```
197 interface Combatant {
198     void attack();
199 }
200 interface Actor {
201     String name();
202     void takeAction();
203 }
204
205 @part class Hero implements Combatant, Actor {
206     public void attack() { println("Swing club!"); }
207     public String name() { return "Milo"; }
208     public void takeAction() {
209         print(name() + " acts: "); // Actor dispatch
210         attack();                 // Combatant dispatch
211     }
212 }
213
214 @part class Wizard implements Combatant, Actor {
215     @link Combatant combatant;
216     @link Actor actor;
217     Wizard(Hero h) { this.combatant = h; this.actor = h; }
218     public void attack() { println("Cast spell!"); }
219 }
220
221 class Demon implements Actor {
222     @link Actor actor;
223     Demon(Actor a) { this.actor = a; }
224     public String name() { return "Diablo"; }
225 }
226
227 Wizard wizard = new Wizard(new Hero());
228 Demon demon = new Demon(wizard);
229 demon.takeAction();
230 // Actor dispatch rooted at Demon
231 // Combatant dispatch rooted at Wizard
232 // prints: Diablo acts: Cast spell!
```

Fig. 2. Composition with multiple composite roots per part. Hero carries distinct composite identities for Combatant (rooted at Wizard) and Actor (rooted at Demon).

3.2 Definitions

Parts is defined over a standard typed object model where classes implement interfaces and method dispatch is defined by the receiver's runtime type.

claim an interface of a part is claimed when a composite links that interface, making the composite its exclusive identity within the part.

component a class nominally implementing one or more interfaces, available for delegation if the class is a part and for forwarding otherwise.

composite a class containing one or more links. A composite nominally implements the interfaces declared on its links.

delegation surface a component's interface hierarchy, excluding any interface marked `@internal` in its `implements` clause.

interface a stateless abstract type that defines a contract (a Java interface).

link an ordinary field annotated with `@link` and typed as an interface. The enclosing class (the composite) delegates the link's declared interface to the component assigned to the link at construction. Links are implicitly `private` and `final`; explicit modifiers are redundant.

part a component annotated with `@part`, which signals compiler plugin processing for open recursion support.

partial delegation a requirement for arbitrary compositions, where a composite may claim only some of a part's delegation surface.

root (of an interface) the linking composite claiming an interface of a part.

When a composite links an interface to a part, the part is directly wired to the composite; it records a *composite identity*: a reference to the composite statically indexed by the interface. The part's self-calls route through the composite indexed by the interface of the called method. The enabling mechanism is *interface-scoped dispatch* (ISD).

A part may itself contain link fields, in which case composite identity propagates transitively. When such a part is wired to an outer composite, that composite becomes the dispatch root for each interface it claims throughout the delegation chain, while interfaces it does not claim retain the roots established by inner composites. This *transitive propagation* continues through a link field only when the field's value is a part and the field's declared interface and the claimed interface are comparable, one a subtype of the other (Figure 6).

Composites must also resolve *diamonds*. When a composite has multiple links that share a common interface, a sharing link must be chosen explicitly via `@link(share=I.class)` at one of the link sites. This determines the leg of the diamond the linking composite forwards through for its implementation of the interface. Diamonds consisting of multiple shared interfaces require the same `share=I.class` to select the sharing links for each interface, where a single link can share multiple interfaces or the interfaces can be shared separately from any of the other links capable of sharing them. Wiring is unaffected: self-calls on common interfaces from other legs of the diamond naturally route through the shared leg.

Two practical extensions, composite-scoped encapsulation (`@internal`) and abstract parts with deferred linkage (`asLink()`), are defined and implemented (Section 4).

Dispatch rule. Traditional object-oriented dispatch resolves a call through the receiver's object identity:

$$\text{call } m() \text{ on } o \longrightarrow \text{vtable}(o, m)$$

Interface-scoped dispatch replaces the single object identity with a per-interface composite identity. A self-call $m()$ in part P where $m \in I$ resolves through the composite identity for I :

$$\text{self-call } m() \in I \text{ in } P \longrightarrow \$\text{selfes}_P[I].m()$$

For self-calls within a part's methods, the dispatch triple is (*part, interface, method*) rather than the conventional (*object, method*). Figure 3 contrasts this per-interface routing with a single shared composite identity. This is *open recursion*: a method's self-calls late-bind to the most-derived receiver. Here the receiver resolves from the interface of the self-call. Forwarding is the closed case: a component's self-calls stay in the component, and the composite never enters the recursion.

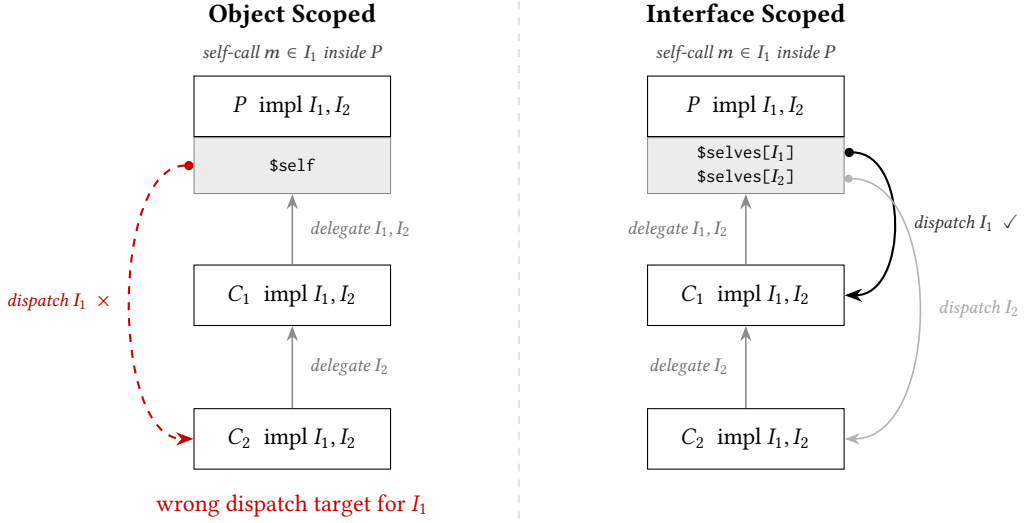


Fig. 3. Object-scoped vs. interface-scoped dispatch. In both panels C_2 (impl I_1, I_2) delegates only I_2 to C_1 (impl I_1, I_2), which delegates both interfaces to part P . *Left*: a self-call $m() \in I_1$ inside P routes through $\$self = C_2$, the wrong composite for I_1 . *Right*: $\$selves[I_1]$ routes to C_1 ; $\$selves[I_2]$ routes to C_2 ; each self-call reaches its correct composite root.

INVARIANT 1. For any part P in a delegation chain for interface I rooted at composite C : a self-call $this.m()$ within P where $m \in I$ dispatches to C .

This invariant presumes each interface has a single composite root. The model maintains this precondition for every composition it admits but does not statically enforce it.

The invariant is established by construction. When composite C establishes a link to part P for interface I , the plugin-generated `$linkPartToSelf` (Figure 6) sets $\$selves_P[I] \leftarrow C$ and recurses into P 's own links, propagating C 's identity for each interface C claims and leaving all other slots unchanged. Each interface of a part resolves through a single $\$selves$ slot. Wiring an outer composite that claims I re-roots this slot to that composite and leaves the slots for interfaces it does not claim unchanged, so the slot holds exactly one root at any time and stabilizes once the outermost composite claiming I is wired; C denotes that final root. Each write occurs during the wiring composite's construction. Wiring happens solely as link field assignment, and the plugin enforces `final` on link fields, so writes are confined to construction: the $\$selves$ slots are never written after the outermost composite claiming I is constructed.

Visibility follows from safe publication. Once the composite root is safely published, the Java Memory Model guarantees that any thread observing it sees all writes that happened-before publication, the wired slots included. The role of `final` here is write-confinement, not the JMM's final-field freeze. Since array-element writes are not covered by freeze semantics, the guarantee rests on write-confinement together with safe publication of the root, not on the finality of the slots.

Self-calls in P where $m \in I$ are rewritten by the plugin to $((I) \$selves[IDX_I]).m()$. For self-calls through the delegation surface, there is no code path in a part method that dispatches through anything other than the indexed slot. Because the slot holds C once the composition is constructed and is not written thereafter, the invariant holds at every point a self-call can execute.

Before wiring, each slot is initialized to this; an unwired part dispatches on itself, consistent with standard object-oriented dispatch. In this initial state, the invariant holds trivially. A full mechanized proof in a Featherweight Java subset is a natural direction for follow-on work (Section 6.4).

3.3 Arbitrary Composition

Mirroring the structural flexibility of standard object composition and forwarding, Parts supports unrestricted topologies. These are expressed as behaviorally acyclic graphs where nodes are parts, composites, or ordinary objects, and edges are link fields. Each edge establishes a connection from a composite to a component through the interface specified by the link.

Partial delegation permits a composite to claim only the interfaces it needs from a part's delegation surface, leaving the remaining interfaces for other composites to claim. This freedom gives rise to the four composition topology primitives shown in Figure 4 and classified over two dimensions: the number of roots a part maintains and the number of sources the composition has. A root is an outermost node claiming one or more interfaces of a part's delegation surface; a source is a node with no incoming link edges.

- (a) **Single-root, single-source.** Composite A implements I_1, I_2 and delegates both to B , which implements both directly. A 's links claim all of B 's delegation surface, so B has one root, A , for both interfaces, and one source. This is the topology typical of decorator chains.
- (b) **Multi-root, single-source.** Composite A implements I_1, I_2 and delegates only I_1 to B , which implements I_1, I_2 directly. Because A does not claim B 's I_2 , I_1 is rooted in A , while I_2 is rooted in B : one source, two roots. A and B in turn may delegate their free interfaces to separate components that participate in another subgraph that likewise decomposes into one of these four topologies. The motivating example discussed previously (Figure 1) is an instance of this topology combined with (a) where B delegates both its I_1 and I_2 to another part, propagating A and B as the part's roots for I_1 and I_2 .
- (c) **Multi-root, multi-source.** Component C implements I_1 and I_2 , where A claims I_1 and B claims I_2 . As in (b), C has different roots for its two interfaces, but here A and B are independent sources. This echoes subject-oriented programming's independent *subjects*, composed with no single subject designated as owner (Section 7). Figure 5 exercises this directly.
- (d) **Cyclic, multi-root, sourceless.** Composite A implements I_1, I_2 and delegates I_1 to B , which implements I_1, I_2 and delegates I_2 back to A . Such cycles are permitted when delegation through the cycle is behaviorally terminal because each traversal proceeds through a distinct interface: I_1 is rooted in A , and I_2 is rooted in B . Cycles over nondistinct interfaces are rejected (Section 4).

Each of the four topologies illustrated takes its reduced form using as few nodes and edges as possible while retaining the essential properties that make the topology distinct. Topologies (a) and (b) each have a single source, but (b) is multi-root; (c) is multi-root and multi-source; (d) is multi-root and sourceless. Topology (d) is structurally cyclic but behaviorally acyclic over distinct interfaces.

These four are exhaustive over the two dimensions. Of the six combinations the dimensions admit, two cannot occur: a single-rooted part forces a single source, and a sourceless graph is necessarily multi-root. This leaves (a)–(d).

Forwarding admits these topologies, so by definition a general delegation model must as well. The topologies therefore provide a basis for verifying whether a delegation model supports arbitrary composition: does its treatment of open recursion remain correct under arbitrary extensions of each topology?

To answer this question, another set of labeled edges may be drawn onto the structural delegation graph to form a second graph in which each part is connected to the composites claiming its interfaces, determined by following the delegation edges backwards along each interface of the part's delegation surface to the outermost composite claiming the interface. This derived identity graph reveals what open recursion under arbitrary composition requires: composite identity assigned per interface.

In all four cases, for each delegated interface, a part acquires the identity of the composite claiming that interface, independent of where execution enters the composition. Invariant 1 holds independently for each interface.

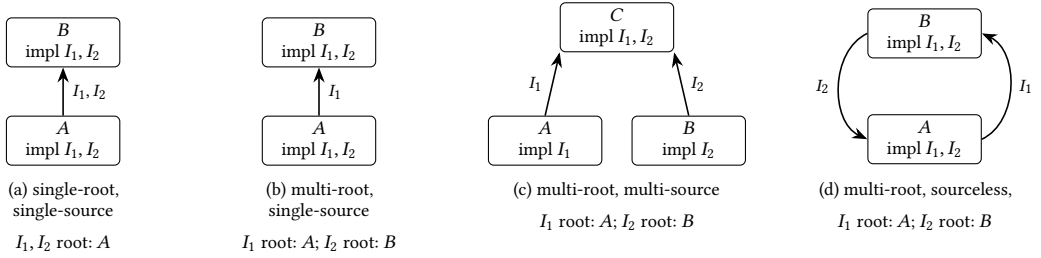


Fig. 4. The four primary composition topologies under partial delegation. Arrows point from a delegating composite to the part it links. (a) and (b) have a single source; (c) has two sources; (d) is sourceless.

```

442 interface Actor { void takeAction(); int timeLimit(); }
443 interface Combatant { int strengthLevel(); }
444
445 @part class BasicHero implements Actor, Combatant {
446     private int elapsedTime = 11;
447     public void takeAction() { println("Attack with strength: " + strengthLevel()); }
448     public int timeLimit() { return 10; }
449     public int strengthLevel() { return outOfTime() ? 0 : 8; }
450     private boolean outOfTime() { return elapsedTime > timeLimit(); }
451 }
452 class PaidSubscriber implements Actor {
453     @link Actor actor;
454     PaidSubscriber(Actor actor) { this.actor = actor; }
455     public int timeLimit() { return 100; }
456 }
457 class SuperSoldierAddon implements Combatant {
458     @link Combatant combatant;
459     SuperSoldierAddon(Combatant combatant) { this.combatant = combatant; }
460     public int strengthLevel() { return combatant.strengthLevel() * 10; }
461 }
462 BasicHero basicHero = new BasicHero();
463 PaidSubscriber subscriber = new PaidSubscriber(basicHero);
464 SuperSoldierAddon addon = new SuperSoldierAddon(basicHero);
465 subscriber.takeAction(); // reaches SuperSoldierAddon's strengthLevel()
466 // PaidSubscriber and SuperSoldierAddon hold no reference to each other.
467 // prints: Attack with strength: 80

```

Fig. 5. Topology (c): two independent composites, PaidSubscriber and SuperSoldierAddon, claim different interfaces of the same BasicHero with no common composite. The call subscriber.takeAction() reaches self-calls through the respective roots: strengthLevel() through SuperSoldierAddon and timeLimit() through PaidSubscriber.

4 Implementation

The implementation is a javac plugin operating on the compiler's AST directly before and after two phases: *Enter* (symbol definition) and *Analyze* (semantic processing).

For the Enter phase, the plugin generates structural scaffolding. Here, @part classes are processed to add the \$selves identity array, the \$linkPartToSelf wiring method, and if the class is abstract, the \$Impl subclass and asLink() factory methods. Composite classes are also processed here to add forwarding methods corresponding with @link delegation. At Analyze, @part classes are processed to rewrite self-calls and to enforce self-preservation rules (Section 4.5). @link field assignment is also rewritten here to invoke wiring, and to enforce @internal.

4.1 Part Transformation

Each part holds a \$selves array where the slots of the array each correspond with an interface on the part's delegation surface. The ordering of the array's slots is compile-time-determined according to the part's class information.

```

487 private final Object[] $selves = {this, ..., this};
488 // after wiring: $selves[IDX_I] = composite identity for interface I
489

```

Array slots are initialized with `this`, so a part's self-calls dispatch on itself by default. Within a part, every call `this.m(args)` where `m` is declared in interface `I` is rewritten to:

```
((I) $selves[IDX_I]).m(args)
```

The cast ensures interface method invocation via the `invokeinterface` instruction, which guarantees a composite receiver will itself route the call correctly within its runtime implementation, regardless of indirection such as with bridge methods.

The ordinal `IDX_I` is assigned per part over its own delegation surface, so it is local to that class: the rewritten self-calls and the generated `$linkPartToSelf` both reside in the part and share its indexing, and no index agreement across separately-compiled classes is required.

```
public void $linkPartToSelf(Object root, Class<?>[] linkScope) {
    // cycle detection
    if (this == root) Internal.reportCycle(this, root, iface);
    // wire the part's 'linkScope' interfaces to 'root'
    outer:
    for(Class<?> linkIface : linkScope) {
        for(int i = 0; i < $INTERFACE_CLOSURE.length; ++i) {
            Class<?> iface = $INTERFACE_CLOSURE[i];
            if (iface == linkIface) {
                this.$selves[i] = root;
                continue outer;
            }
        }
        throw new DelegationLinkageError("Unimplemented linked interface: " + linkIface);
    }
    // recursively call $linkPartToSelf to rewire the part's related links to 'root'
    Class[] scopeLinkField1 = Internal.widest($LINK_FIELD_1, linkScope);
    if (scopeLinkField1.length != 0 && this.linkField1 instanceof $PartClass) {
        (($PartClass)this.linkField1).$linkPartToSelf(root, scopeLinkField1);
    }
    // likewise for the linkField2, etc.
}
```

Fig. 6. Generated `$linkPartToSelf` for a part class. `Internal.widest` returns the widest of the two narrowest-first ordered arrays, or empty array if they are not comparable.

4.2 Composite Transformation

A composite nominally implements every interface claimed by its links. For each method of a claimed interface that the composite does not implement directly, the plugin generates a delegating method that forwards through the corresponding link field:

```
R m(args) { return link.m(args); }
```

In composite classes, each link field assignment `this.link = part` is transformed to wire part at link assignment via `part.$linkPartToSelf(this, I.class)` where the composite (`this`) is assigned to the part's `$selves` slot corresponding with the link's declared interface `I`. Then, if the part itself links the same interface to another part, it recursively calls `$linkPartToSelf` to rewire the link's part to the calling composite (Figure 6).

4.3 Abstract Part Support

A part may be abstract, deferring some interface methods to the composite. Linking an abstract part obligates the composite to implement those methods or to be declared abstract. When enforceable at compile time, the plugin generates a delegating method for every interface method the part implements and none for a method it leaves abstract. Missing those methods, a concrete composite is flagged by the compiler to either implement them directly or declare itself abstract. This is the compositional analog of the rule that a concrete class must implement its inherited abstract methods.

To make an abstract part instantiable for linking, the compiler generates a private `$Impl` inner subclass with stubs that throw `DelegationLinkageError`, and an `asLink()` factory per constructor:

```
private static final class $Impl extends AbstractClass {
    $Impl(/* AbstractClass ctor params */) { super(...); }
    @Override public R m(/* params */) {
        throw new DelegationLinkageError("Abstract method called");
    }
}
public static Foo asLink(/* ctor params */) { return new $Impl(...); }
```

Once wired, the stubs are unreachable for a statically configured abstract part. Self-calls dispatch through `$selves` to the composite, where the compile-time obligation guarantees an override. The stubs enforce the runtime configured case when the linking composite does not override any of the part's abstract methods. The stubs also serve as a runtime guard for direct abstract method calls on an `asLink()` instance.

4.4 Default Methods

If left unmitigated, a default method runs with the part as receiver, rather than the composite, when neither the composite nor the part overrides it, resulting in incorrect self-call dispatch.

No bytecode instruction directly invokes a default with a receiver other than the current `this`, but a method handle bound to an invokedynamic call site works around this limitation. When a part does not override a default, the plugin generates an override that invokes the interface's default implementation in this way, with the receiver taken from the interface's slot in `$selves`, so the default's self-calls dispatch through that receiver. Similarly, an explicit `I.super.m()` invocation within a part is rewritten the same way.

```
interface Foo {
    void bar();
    default void foo(P p) { bar(); } // self-call
}
@part class FooPart implements Foo {
    public void bar() { ... }
    // generated
    // calls Foo.super.foo() with composite in $selves as receiver
    public void foo(P p) {
        invokedynamic Foo::foo((Foo) $selves[IDX_Foo], p);
    }
}
```

The site's bootstrap links it once to a fixed target (the default implementation), with the receiver as its only varying argument. Being monomorphic, it is inlined by the JIT to the same type of $O(1)$ array-indexed invocation as in Section 5.

4.5 Safety Guarantees

Cycle detection. If a part is wired to itself directly or transitively over the same interface, `DelegationLinkageError` is thrown at assignment time during the recursive `$linkPartToSelf()` call (Figure 6).

@internal enforcement. `@internal` may annotate an interface method or an interface type literal in a class's `implements` clause, with distinct effects. A method annotated with `@internal` is accessible only within the composite, the same intent behind `protected` in implementation inheritance, but more cleanly scoped for composition. A type literal in a class's `implements` clause annotated with `@internal` removes that interface from the class's delegation surface: no composite may claim it via a link field.

Self-preservation. Inside a part, this use is guarded. Field access, private calls, and callbacks through the part's own link fields (the compositional analog of `super`) are all permitted. Passing or assigning this as the concrete part type is not; that is a compile error, since a caller holding the bare part could invoke methods outside the composite dispatch path and violate Invariant 1. Such this references must either be interface-typed or `Object`-typed, resolving via ISD to the composite claiming the interface or the part itself, guaranteeing the recipient reaches the correct overrides.

A bare self-call to a method either belonging to multiple interfaces or shared via a common interface between multiple unrelated interfaces is ambiguous and results in a compile error. The self-call's `this` receiver must be qualified via casting to the intended interface (Figure 7). In this way parts can target separate unrelated overrides of the same method. This behavior is vital for multi-source structures of topology (c) (§3.3), common with subject-oriented designs.

```

638 interface Z { String z(); }
639 interface A extends Z { String a(); }
640 interface B extends Z { String b(); }
641 @part class AB implements A, B {
642     public String a() { return ((B)this).z() + " " + ((A)this).z(); }
643     public String b() { return ((A)this).z() + " " + ((B)this).z(); }
644     public String z() { return "AB.z"; }
645 }
646 class AImpl implements A {
647     @link A a;
648     AImpl(A a) { this.a = a; }
649     public String z() { return "AImpl.z"; }
650 }
651 class BImpl implements B {
652     @link B b;
653     BImpl(B b) { this.b = b; }
654     public String z() { return "BImpl.z"; }
655 }
656 AB ab = new AB();
657 AImpl a = new AImpl( ab );
658 BImpl b = new BImpl( ab );
659 println(a.a()); // "BImpl.z AImpl.z"
660 println(b.b()); // "AImpl.z BImpl.z"

```

Fig. 7. Self-calls to overlapping method `z()` in interfaces `A` and `B` must be cast-qualified to the intended interface. Here, part `AB` reaches both implementations of `z()` by selecting the target interface with casting.

5 Evaluation

5.1 Dispatch Performance

The dispatch path for a self-call through a composite is:

- (1) Load `$selves[IDX_I]`, an array-indexed load with compile-time constant index
- (2) Cast to `I`
- (3) `invokeinterface m`, standard JVM virtual dispatch

This is $O(1)$. The parallel to `vtable` dispatch is direct. A `vtable` call is an indexed slot load followed by an indirect call. A self-call within a part is therefore on equal footing with a `vtable` call.

Benchmark. We verify this empirically using JMH [32] in average-time mode. Each scenario calls `compute()` through a composite root at chain depth d (1–6). The *baseline* traverses d `@link` delegation hops to a leaf that returns directly, with no self-call. The *concrete* variant traverses the same hops but the leaf issues a self-call (`$selves[IDX_I].scale()`) from a concrete part method, dispatching through the composite root. The *default* variant issues the same self-call from an inherited interface default method the leaf does not override, routed through the generated `invokedynamic` override of Section 4.4. The delta (concrete minus baseline) isolates the cost of the self-call dispatch mechanism alone. All runs: 10 JVM forks, 5 warmup iterations, 10 measurement iterations per fork (100 data points per cell); errors are $\pm$ half-width of the 99.9% confidence interval. A separate single-dispatch calibration run (`invokeinterface` on a monomorphic call site, same harness) yields 0.43 ns/op; the delta values in Table 1 are therefore roughly 1–2 $\times$ a single virtual dispatch.

Choice of baseline. The delta is measured against Parts's own forwarding path, which establishes the baseline: that path is semantically equivalent to Kotlin by and Lombok @Delegate (Section 7), so the delta prices exactly the self-call routing those mechanisms cannot express, against their own dispatch cost. A cross-system benchmark against other composition models would price differences in implementation strategy rather than the self-call dispatch this delta isolates, so that comparison is analytic (Section 7).

Table 1. Self-call dispatch cost by delegation depth (JMH, average-time mode; ns/op, $\pm 99.9\%$ CI half-width). *Baseline* traverses the delegation hops with no self-call. *Concrete* and *Default* add one self-call through the composite, issued from a concrete part method and from an inherited interface default (via `invokedynamic`) respectively. The delta (Concrete minus Baseline) isolates the self-call dispatch cost over plain delegation. Values are rounded from the measured means and intervals. The delta combines the Baseline and Concrete cells (difference of means, half-widths in quadrature) and may differ from the rounded column difference by 0.01 ns.

Depth	Baseline (ns)	Concrete (ns)	Default (ns)	Delta (ns)
1	0.62 ± 0.02	1.16 ± 0.03	1.18 ± 0.03	0.55 ± 0.03
2	0.91 ± 0.03	1.56 ± 0.04	1.55 ± 0.04	0.65 ± 0.05
3	1.18 ± 0.03	1.93 ± 0.05	1.92 ± 0.05	0.75 ± 0.06
4	1.64 ± 0.03	2.30 ± 0.06	2.23 ± 0.06	0.66 ± 0.07
5	1.93 ± 0.06	2.70 ± 0.06	2.76 ± 0.07	0.77 ± 0.09
6	2.37 ± 0.06	3.21 ± 0.09	3.23 ± 0.08	0.84 ± 0.11

Across depths 1–6 the delta remains bounded, on the order of a single virtual dispatch (the 0.43 ns calibration above), consistent with one array-indexed load (`$selves[IDX_I]`) followed by `invokeinterface`. The self-call routes through a dynamically wired `$selves` slot, so its receiver is not statically known and the dispatch does not fold away: the delta is one virtual dispatch, with the array indexing on top (confirmed free below). The $O(1)$ claim is structural: one dispatch per self-call, independent of depth, with the per-call cost pinned by the isolation measurement below. The small upward drift in the delta across depths (0.55 to 0.84 ns) stays well under a second dispatch and plausibly reflects less complete inlining of the dispatch site within the larger compiled method at greater depth. The *default* column tracks the *concrete* column within $\pm 3\%$ at every depth, with the sign of the difference varying. The two are indistinguishable at this resolution. The interface-default template, dispatched through `invokedynamic`, carries no measurable cost over the concrete part template, confirming the equivalence claimed in Section 4.4. The comparison is steady-state: the one-time bootstrap that links each `invokedynamic` site is absorbed during warmup and is not measured here.

Isolating the array index. The depth-series delta is essentially one virtual dispatch. To confirm the `$selves` array indexing adds nothing on top, we isolate the dispatch. Holding a target method non-inlinable so a real call occurs, we compare three depth-0 forms over the same receiver: a `vtable` call (`invokevirtual`), the `invokeinterface` it compiles to, and the plugin's emitted form (`((I) $selves[IDX_I]).m()` (an `Object[]` element load, a cast, and `invokeinterface`). Over 10 JVM forks (JDK 21) the three are statistically indistinguishable. The ISD form differs from the bare `invokeinterface` it emits by 0.03 ± 0.09 ns, and from `invokevirtual` by 0.10 ± 0.05 ns, both within the fork-to-fork variance of the measurement. The array-element load is not separately observable, therefore the self-call costs one virtual dispatch and no more. This is the empirical content of the `vtable`-equivalence claim.

Construction cost. The result above is the steady-state call cost; wiring is paid once, at construction. Building a composite invokes `part.linkPartToSelf` (Figure 6) at each link assignment involving a part, which sets the part's relevant `$selves` slots and recurses through the part's own link fields, propagating composite identity for each interface the composite claims, where the interface is comparable to the link field's interface. Each pass does constant work per part per claimed interface (an identity check for cycle detection, a constant-time scope comparison, and no allocation). Wrapping an already-linked composite re-propagates its subtree, so total wiring cost grows with delegation depth, which is small in practice. None of this lies on the dispatch path, so the per-call cost is unchanged.

5.2 Example: Teaching Assistant

The following example exercises diamond dispatch resolution and self-call routing through the composite. A teaching assistant (TA) is both a student and a teacher; both roles share a common Person base.

```

interface Person {
    String name();
    String title();
    String titledName();
}

interface Student extends Person { ... }
interface Teacher extends Person { ... }
interface TA extends Student, Teacher { ... }

@part abstract class PersonPart implements Person {
    private final String name;
    public PersonPart(String name) { this.name = name; }
    public String name() { return name; }
    public String titledName() { return title() + " " + name(); }
}

@part class StudentPart implements Student {
    @link Person person;
    public StudentPart(Person p) { this.person = p; }
    public String title() { return "Student"; }
}

@part class TeacherPart implements Teacher {
    @link Person person;
    public TeacherPart(Person p) { this.person = p; }
    public String title() { return "Teacher"; }
}

@part class TaPart implements TA {
    @link(share=Person.class) Student student;
    @link Teacher teacher;
    public TaPart(Student student) {
        this.student = student;
        this.teacher = new TeacherPart(student);
    }
    public String title() { return "TA"; }
}

```

```
}  
  
TA ta = new TaPart(new StudentPart(PersonPart.asLink("Milton")));  
ta.titledName(); // "TA Milton"
```

With forwarding, the shared `Person` methods must each be resolved by hand, and `titledName()` still calls `title()` on the delegate's own `this`. `Parts` resolves both problems.

`PersonPart` holds the template once. Its self-call to `title()` dispatches through the composite at each level of the delegation chain.

`titledName()` is defined once in `PersonPart` and overridden nowhere. Diamond resolution is handled statically: `@link(share=Person.class)` designates `student` as the forwarding link for `Person`, `TaPart` transitively claims `Person` throughout the composite.

6 Discussion

6.1 Partial delegation and part identity

Partial delegation is a requirement for arbitrary composition, where a composite may claim only some of a part's delegation surface. *Multi-rooted parts* result from this condition: a part's interfaces may be rooted at two or more distinct identities, including the part itself and one or more composites, via any combination of the multi-rooted topologies in Section 3.3.

The possession example in Section 3 demonstrates an extended version of topology (b): `Demon` claims only `Actor` from `Wizard`. Through delegation propagation, `Hero` consequently has separate roots for `Actor` and `Combatant`, making it a multi-rooted part through a single source graph, `Demon`. The `PaidSubscriber/SuperSoldierAddon` example (Figure 5) demonstrates topology (c): multiple roots within a multi-source composition.

This multi-rootedness makes a part's identity context-sensitive. References to this, whether implicit or explicit, resolve according to the context type of the reference. When that type is a claimed interface, ISD resolves this to the claiming composite. Otherwise, this remains the part itself. This is a direct resolution of the Object Schizophrenia problem mentioned in Section 1.

6.2 Language Design Implications

`Parts` resolves the principal deficiencies of implementation inheritance:

- **Fragile base class** [2]: a composite's dependency surface is the interface contracts it explicitly claims, not a components's internal implementation; changes to its encapsulated internals do not propagate. Partial delegation completes the isolation: a composite is unaffected by changes to interfaces it does not claim.
- **Forced IS-A semantics**: reuse does not require a type relationship; the composite is a subtype of the interface, not of the component's implementation.
- **Hierarchy-bound**: implementation inheritance is rooted in a single runtime object and therefore supports only single-source topologies; a component cannot participate in multiple, independently rooted composites. `Parts` permits arbitrary topologies by assigning composite identity per interface.
- **The Object Schizophrenia problem** [4]: a hazard with forwarding where self-calls remain within the component rather than routing through the composite; resolved by ISD.
- **Diamond ambiguity**: explicit `@link(share=I.class)` resolution; unresolved diamonds are compile errors, not silent merges.
- **Encapsulation breach**: a composite may implement a subset of its parts' interfaces, those it does not implement are absent from its API by construction. `@internal` strengthens this further: a class may declare selected interfaces as internal to its implementation and

excluded from its delegation surface, preventing the interfaces from being claimed by a composite.

- **Abstract behavior:** abstract parts with deferred linkage (`asLink()`) preserve the template-method capability of abstract classes without coupling their implementation and hierarchy.

Implementation inheritance’s power is open recursion. Its fragility is the surface that power ranges over—every accessible member. Parts retains open recursion but bounds the surface to the declared interface contract. Self-calls still reach the composite’s overrides, while the internals that make base classes fragile are never exposed.

Implementation inheritance and Parts are, essentially, two bindings of the same open recursion: implementation inheritance binds it to the class hierarchy, where a subclass IS-A its superclass and inherits its entire implementation surface, while Parts binds it to a held object and its delegation surface, where the composite HAS-A its parts and IS-A only its interfaces. The concept that inheritance and open recursion are separable is Cook’s [6, 7], and the modeling of inheritance as composition is Bracha and Cook’s [5]. Cook’s model threads self as an explicit parameter; Parts wires composite identity per-interface at construction.

6.3 Concurrency

A constructed composition is safe to share across threads. Each interface’s `$selves` slot is written only while its final composite root (Invariant 1) is under construction. Once that root is safely published the slot is never written again, so the JMM makes all wired state visible to any thread observing the root, with no synchronization on the dispatch path. The guarantee is scoped to a single composition, resting on the one-root invariant of Section 6. Parts may hold mutable state under standard Java rules and dispatch adds no synchronization points.

6.4 Future Work

Per-interface composite roots as a general primitive. Assigning dispatch identity per interface at construction is, perhaps, a primitive that may be applied in areas beyond this paper’s focus. Problems currently addressed through dynamic dispatch, AOP weaving, or collapsed component hierarchies may have cleaner solutions within it. The extent of that solution space is an open question.

Formal verification. A mechanized proof of Invariant 1 using small-step operational semantics would provide rigorous structural verification beyond the constructive proof sketch outlined in Section 3. Because the Parts model maps cleanly onto a core calculus, a modified subset of Featherweight Java serves as a natural starting point for formalizing these type-safety and execution guarantees.

Link timing and mutability. Through immutable links, composite identity is finalized at construction. This read-only guarantee is what makes sharing lock-free (Section 6): stable wiring needs no synchronization on the dispatch path. Two relaxations of the rule, opt-in *lazy* and *mutable* links, are worth exploring, and they go in opposite directions.

A *lazy* link changes only the timing. Backed by the stable-value facility of recent JDK releases (Project Leyden), it wires on first use instead of at construction, yet stays written once and effectively final, so the guarantee holds while link initialization is deferred. A *mutable* link lifts the restriction. Mutating link assignments become legal, restoring the only aspect of conventional forwarding Parts disallows (runtime reconfiguration) but sacrifices the guarantee and under the same thread-safety obligations as a regular mutable field. Both address restrictions of the current model, not limits of

it. Whether a mutable link can also keep the guarantee, by confining rewiring to safe-publication points, is an open question.

Domain model and persistence integration. IS-A relationships in normalized relational schemas map naturally to the Parts model: each type owns its interface and its data independently, composed at construction with open recursion available across the composed object. Much of the pressure to denormalize comes from the object model: ORM frameworks typically fall back on a single-table (table-per-hierarchy) strategy, forcing sparse, null-heavy tables, or a joined-table (table-per-subclass) strategy that keeps data normalized but still forces a choice between an inheritance hierarchy that couples the types and hand-wired composition that gives up open recursion. Parts needs neither: interface-scoped dispatch keeps the types independent while preserving open recursion, letting the domain model mirror the normalized schema directly. How this mapping is realized inside a persistence framework is an open question.

7 Related Work

While Parts assumes static typing, this section also reviews dynamically-typed models where the structural comparisons remain conceptually instructive.

Forwarding. Kotlin’s by keyword [29], Lombok’s @Delegate [30], Scala 3’s export keyword [28], and Go’s struct embedding [31] forward to a supplied component. In all cases, self-calls resolve in the component’s scope alone; they do not dispatch to the composite.

Scala traits / abstract type members. Odersky and Zenger [11] introduced statically-typed mixin composition with abstract type members. Scala’s with clause linearizes traits into the target class at compile time, so self-calls dispatch through the outermost class in the linearized hierarchy. This gives open recursion, but it also fuses the component into the class, so the trait’s implementation is resolved at compile time, not supplied as an independent component at construction. A component implementation therefore cannot be chosen or replaced, and a single instance cannot be shared across composites. The fusion and the open recursion are inseparable: linearization is the single mechanism behind both. This fusing of the component into the class is the standard recipe for precisely-typed mixin systems [12]. Parts’ interface-scoped dispatch shows the fusion was never necessary: it obtains the same open recursion without abandoning independent components.

Mixins. Bracha and Cook [5] formalized mixins as parameterized abstract subclasses. Mixins extend through linearization: the mixin is applied to a chain at class definition time, not supplied as an independent component at construction. Self-calls dispatch through this in the linearized result.

Traits. Schärli et al. [8] proposed traits as stateless, composable units of behavior. Trait composition is a compile-time operation: methods are flattened into the target class, not supplied as independent components at construction. Self-calls dispatch through this in the composed class.

Typed self-recursion calculi. Chugh’s ISOLATE [13] is a statically-typed calculus of functional objects that types a pattern of mixin composition with open recursion through self, without an explicit fixpoint. Its units of composition are *premethods*—functions defined apart from a host object and later mixed into it—so a method may be “defined independently of its host object,” but the object itself is still assembled by fusion rather than supplied as an independent late-bound component. ISOLATE sits on the mixin side of the trade: open recursion without independent components, as in recent precisely-typed mixin systems [12].

Multiple inheritance. Stroustrup [14] introduced virtual base classes and virtual inheritance in C++ to handle diamonds. The result is a single linearized object with a merged vtable; components are not late-bound objects.

Dynamic single inheritance. Kniesel’s DARWIN [15, 16] is a statically typed delegation-based model of single implementation inheritance: a base object is supplied at run-time through a delegatee field, its self-calls routing through the inheriting subtype. Being single inheritance, it admits at most one delegating (or forwarding) field per class [16, §6.5]; multiple delegation is *deliberately* excluded [16, §6.5.1], and a single receiver typed at one interface admits no multi-root composition [16, §5.5.4]. DARWIN achieves open recursion, but only over a single inherited base (topology (a)), sacrificing arbitrary composition. Parts’ ISD supplies open recursion together with multiple delegation—required for arbitrary composition—by assigning identity per interface at construction.

Deep delegation. DelphJ [17] is a Java-based research language that augments its MorphJ metaprogramming DSL with delegation features (*access-paths*) as a means for open recursion. Its runtime-supplied *subobjects* generally support late-bound self-calls in single-rooted compositions. Its caller-based strategy using access-paths introduces substantial practical costs and, like Kniesel’s DARWIN, sacrifices arbitrary composition. As the authors point out, their “deep delegation” incurs a considerable runtime penalty building and merging access paths for stack-held references on field reads, and traversing them via library-based lookup on every self-call, spreading dispatch costs throughout the runtime. Crucially, the access-path strategy is fundamentally incapable of arbitrary composition. For instance, the model cannot correctly dispatch cross-interface self-calls to methods common to separately claimed interfaces (Figure 7); this case spans topologies (b)–(d). Similarly, access-paths cannot support cross-interface self-calls at all with topology (c). Its composition is therefore generally limited to topology (a) (§3.3). Parts inverts the access-path strategy: composite identity is fixed in the component rather than reconstructed by the caller on every access. This single design choice is what makes self-dispatch on par with vtable performance, and perhaps most importantly, is what makes delegation under arbitrary composition possible.

Incomplete objects. Incomplete Featherweight Java (IFJ) [18] is a statically typed, delegation-based calculus in which *incomplete objects* are completed at run time by complete objects. Like Parts, it composes independent runtime objects with open recursion under static typing. Its delegation, however, is structural and per method, and binds a single *this* to the sender throughout, so a component cannot root different interfaces at different composites. Its composition is therefore confined to topology (a) (§3.3), the single-rooted, single-source chain. Parts binds each interface to a specific delegate through @link, assigning composite identity per interface, which admits the multi-rooted and multi-source topologies (b)–(d) that partial delegation induces with ordinary forwarding.

Family polymorphism. gbeta [22] (formalized by Ernst, Ostermann, and Cook [20]), CaesarJ [23], Object Teams/Java [24], and J& [21] target extensible class families, not independent components. Here, self-calls dispatch correctly within a family, but the object structure is defined statically by the family type, its members instantiated internally and not supplied independently: open recursion without independent components.

Delegation Layers. Ostermann’s delegation layers [19] scale delegation from single objects to whole collaborations, composed at runtime, with operations routing to the composite collaboration rather than a single layer (“composition consistency”). But the composed unit is a collaboration, not an individual object, and its participants are nested classes redirected through the enclosing instance,

not independently supplied: open recursion at collaboration granularity, without independent components and arbitrary composition (no per-component partial delegation that produces multi-rooted and multi-source topologies).

Prototype-based delegation. Lieberman [3] introduced *open delegation*: an object may delegate unanswered messages to a prototype, with *self* remaining the original receiver throughout the chain. Self [9] realized this in a working language. These systems provide open recursion with late-bound components, but delegation is total and object-granular: a prototype supplies all behavior the delegating object does not, and cannot be scoped to a subset of interfaces, so neither partial delegation nor the multi-rooted topologies it induces (§3.3) are expressible. Static type guarantees are absent by design. Cecil [10] is the statically-typed point in this space. A classless (prototype) language with a type system that separates subtyping from code inheritance, it “does not incorporate dynamic inheritance, one of the most interesting features of Self,” offering predicate objects as a “more structured but more restricted alternative” and attaching behavior only to statically-named objects rather than the run-time-supplied components at issue here.

Dynamic composition. Newspeak [25] makes all names late-bound and injects classes as first-class constructor arguments, enabling runtime reconfiguration and wiring of module objects. Its open recursion is that of mixin inheritance [5]: every class is a mixin applied to a superclass, fused into a single object; composed module objects are held by reference and reached by an ordinary message send, so self-calls do not route across the composition. Static type checking is absent by design.

Subject-oriented programming. Subject-oriented programming [26, 27] composes independent *subjects*: each is a complete, self-contained view of a shared domain. Subjects are merged through externally specified *composition rules* (e.g., merge, override) rather than a single global object model, explicitly to support decentralized, unanticipated composition without a designated owning subject. This targets the same multi-source structure as topology (c) (§3.3): no subject is required to be the sole root of a composed result, and independently developed subjects can be composed later without modifying the ones already in place. This was the theory proposed by the authors. But this system was never realized. Instead, every major implementation built by the original IBM research team ultimately collapsed subjects down into conventional, single-runtime-object systems at compile time or load time, giving up independent runtime objects. Additionally, neither the model nor its published implementations enforce static typing across the composed result [27]. Parts covers the multi-source composition the authors originally aimed for—topology (c)—at the granularity of an individual runtime object, under a statically-checked interface contract.

Novelty. No system surveyed above provides independent runtime components, arbitrary composition, and open recursion together. The models closest to Parts, IFJ, DARWIN, and DelphJ, achieve independent components and open recursion but not arbitrary composition: all are confined to the single-rooted topology, their dispatch mechanisms unable to support the full range of ordinary forwarding. IFJ binds a single *this* by construction, and DARWIN deliberately excludes multiple delegation for stated performance concerns [16, §6.5.1]. DelphJ comes closest: it permits multiple delegation, but its access-path dispatch cannot resolve the cross-interface cases that topologies (b)–(d) require, and it carries substantial runtime penalties its own evaluation acknowledges (§7). What these models set aside, Parts shows is practical with interface-scoped dispatch, achieving all three properties without compromise at vtable-equivalent dispatch cost.

8 Conclusion

The advice to favor object composition over class inheritance came with an unspoken cost: composition gives up the open recursion that made inheritance practical. Parts removes that cost. By

assigning composite identity per interface at construction, it shows that open recursion need not be tied to inheritance's single runtime object; the two can be separated without compromise. The result is a general model that retains the essential properties of simple forwarding—independent runtime components and arbitrary composition topologies—while adding open recursion at vtable speed.

References

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] L. Mihajlov and E. Sekerinski. A study of the fragile base class problem. In *Proceedings of ECOOP*, LNCS 1445, pages 355–382, 1998.
- [3] H. Lieberman. Using prototypical objects to implement shared behavior in object-oriented systems. In *Proceedings of OOPSLA*, pages 214–223, 1986.
- [4] K. C. Sekharaiah and D. Janaki Ram. Object schizophrenia problem in modeling is-role-of inheritance. In *Proceedings of the ECOOP Workshop on Inheritance*, Málaga, Spain, 2002.
- [5] G. Bracha and W. Cook. Mixin-based inheritance. In *Proceedings of OOPSLA/ECOOP*, pages 303–311, 1990.
- [6] W. Cook and J. Palsberg. A denotational semantics of inheritance and its correctness. In *Proceedings of OOPSLA*, pages 433–443, 1989.
- [7] W. R. Cook, W. Hill, and P. S. Canning. Inheritance is not subtyping. In *Conference Record of POPL*, pages 125–135, 1990.
- [8] N. Schärli, S. Ducasse, O. Nierstrasz, and A. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP*, LNCS 2743, pages 248–274, 2003.
- [9] D. Ungar and R. B. Smith. Self: The power of simplicity. In *Proceedings of OOPSLA*, pages 227–242, 1987.
- [10] C. Chambers. The Cecil language: Specification and rationale. Technical Report UW-CSE-93-03-05, University of Washington, 1993.
- [11] M. Odersky and M. Zenger. Scalable component abstractions. In *Proceedings of OOPSLA*, pages 41–57, 2005.
- [12] A. Fan and L. Parreaux. super-charging object-oriented programming through precise typing of open recursion. In *Proceedings of ECOOP*, LIPIcs 263, pages 11:1–11:28, 2023. DOI: [10.4230/LIPIcs.ECOOP.2023.11](https://doi.org/10.4230/LIPIcs.ECOOP.2023.11).
- [13] R. Chugh. ISOLATE: A type system for self-recursion. In *Proceedings of ESOP*, LNCS 9032, pages 257–282, 2015. DOI: [10.1007/978-3-662-46669-8_11](https://doi.org/10.1007/978-3-662-46669-8_11).
- [14] B. Stroustrup. Multiple inheritance for C++. In *Proceedings of the EUUG Spring Conference*, 1987.
- [15] G. Kniesel. Type-safe delegation for run-time component adaptation. In *Proceedings of ECOOP*, LNCS 1628, pages 351–366, 1999. DOI: [10.1007/3-540-48743-3_16](https://doi.org/10.1007/3-540-48743-3_16).
- [16] G. Kniesel. *Dynamic Object-Based Inheritance with Subtyping*. PhD thesis, University of Bonn, 2000.
- [17] P. Gerakios, A. Biboudis, and Y. Smaragdakis. Forsaking inheritance: Supercharged delegation in Delphi. In *Proceedings of OOPSLA*, pages 233–252, 2013.
- [18] L. Bettini, V. Bono, and B. Venneri. Delegation by object composition. *Science of Computer Programming*, 76:992–1014, 2011. DOI: [10.1016/j.scico.2010.04.006](https://doi.org/10.1016/j.scico.2010.04.006).
- [19] K. Ostermann. Dynamically composable collaborations with delegation layers. In *Proceedings of ECOOP*, LNCS 2374, pages 89–110, 2002.
- [20] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *Conference Record of POPL*, pages 270–282, 2006.
- [21] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *Proceedings of OOPSLA*, pages 347–366, 2006.
- [22] E. Ernst. Family polymorphism. In *Proceedings of ECOOP*, LNCS 2072, pages 303–326, 2001.
- [23] I. Aracic, V. Gasiunas, M. Mezzini, and K. Ostermann. An overview of CaesarJ. In *Transactions on Aspect-Oriented Software Development I*, LNCS 3880, pages 135–173, 2006.
- [24] S. Herrmann. Object Teams: Improving modularity for crosscutting collaborations. In *Proceedings of NetObjectDays*, LNCS 2591, pages 248–274, 2002.
- [25] G. Bracha, P. von der Ahé, V. Bykov, Y. Kashai, W. Maddox, and E. Miranda. Modules as objects in Newspeak. In *Proceedings of ECOOP*, LNCS 6183, pages 405–428, 2010.
- [26] W. Harrison and H. Ossher. Subject-oriented programming (a critique of pure objects). In *Proceedings of OOPSLA*, pages 411–428, 1993.
- [27] H. Ossher, M. Kaplan, W. Harrison, A. Katz, and V. Kruskal. Subject-oriented composition rules. In *Proceedings of OOPSLA*, pages 235–250, 1995.
- [28] EPFL. Exports — Scala 3 reference. <https://docs.scala-lang.org/scala3/reference/other-new-features/export.html>, 2024.
- [29] JetBrains. Delegation — Kotlin programming language. <https://kotlinlang.org/docs/delegation.html>, 2024.

[30] Project Lombok. @Delegate — Lombok features. <https://projectlombok.org/features/Delegate>, 2024.

[31] The Go Authors. Embedding — Effective Go. https://go.dev/doc/effective_go#embedding, 2024.

[32] A. Shipilev and contributors. JMH: Java Microbenchmark Harness. <https://github.com/openjdk/jmh>, 2013–2024.

[33] Anonymous. manifold-parts: reference implementation. Artifact; repository URL omitted for double-blind review, 2026.